

CPSC 304 Project Cover Page

Milestone #: 1

Date: October 14, 2024

Group Number: 75

Name	Student Number	CS Alias (Userid)	Preferred E-mail Address
Wendy Phung	76185974	z1l1m	ptat1907@gmail.com
Nilasha Majumdar	55099899	nilasha1	nilashamajumdar2021@gmail.com
Victoria Baek	40749855	c4g4x	vickybaek20@gmail.com

By typing our names and student numbers in the above table, we certify that the work in the attached assignment was performed solely by those whose names and student IDs are included above. (In the case of Project Milestone 0, the main purpose of this page is for you to let us know your e-mail address, and then let us assign you to a TA for your project supervisor.)

In addition, we indicate that we are fully aware of the rules and consequences of plagiarism, as set forth by the Department of Computer Science and the University of British Columbia

1. Project Description:

This project focuses on developing a database system for animal shelter management, which includes tracking animal profiles, adoption processes, shelter operations, and donor and volunteer information. The system models various aspects of shelter management, such as branch locations, animal assignments to cages, veterinary services, and supplies management. By efficiently organizing and updating records, the database ensures smooth operations and effective care for the animals in the shelter.

2. Repository Link:

https://github.students.cs.ubc.ca/CPSC304-2024W-T1/project_c4g4x_u7n5u_z1l1m

3. Schema Differences:

- **Field Length Increase:** The **VARCHAR** fields in many attributes were increased from 20 to 255 characters. This change allows for more flexibility in storing longer strings such as names, addresses, and descriptions.

Before:

```
CREATE TABLE Donor (  
  donor_ID INTEGER,  
  name VARCHAR(20),  
  donated_item VARCHAR(20),  
  PRIMARY KEY (donor_ID)  
);
```

After:

```
CREATE TABLE Donor (  
  donor_ID INTEGER PRIMARY KEY,  
  donor_name VARCHAR(255),  
  donated_item VARCHAR(255)  
);
```

- **Naming Convention:** Attribute's name were modified to be more specific and consistent. For example:

- o **name** in **Donor** table becomes **donor_name**

Before:

```
CREATE TABLE Donor (  
  donor_ID INTEGER,  
  name VARCHAR(20),  
  donated_item VARCHAR(20),
```

University of British Columbia, Vancouver
Department of Computer Science

```
PRIMARY KEY (donor_ID)
);
```

After:

```
CREATE TABLE Donor (
donor_ID INTEGER PRIMARY KEY,
donor_name VARCHAR(255),
donated_item VARCHAR(255)
);
```

- o name, role in Volunteer table become volunteer_name, volunteer_role

Before:

```
CREATE TABLE Volunteer (
volunteer_ID INTEGER,
name VARCHAR(20),
role VARCHAR(20),
started_date DATE,
branch_city VARCHAR(20) NOT NULL,
branch_province VARCHAR(20) NOT NULL,
PRIMARY KEY (volunteer_ID),
FOREIGN KEY (role) REFERENCES Role (role),
FOREIGN KEY (branch_city, branch_province) REFERENCES Branch (city,
province)
ON DELETE NO ACTION ON UPDATE CASCADE
);
```

After:

```
CREATE TABLE Volunteer (
volunteer_ID INTEGER PRIMARY KEY,
volunteer_name VARCHAR(255),
volunteer_role VARCHAR(255),
started_date DATE,
branch_city VARCHAR(255) NOT NULL,
branch_province VARCHAR(255) NOT NULL,
FOREIGN KEY (volunteer_role) REFERENCES Role (volunteer_role),
FOREIGN KEY (branch_city, branch_province) REFERENCES Branch (city,
province)
ON DELETE CASCADE);
```

- o **name** in **Vet** table becomes **vet_name**

Before:

```
CREATE TABLE Vet (  
  vet_ID INTEGER,  
  name VARCHAR(20),  
  clinic VARCHAR(50),  
  PRIMARY KEY (vet_ID)  
);
```

After:

```
CREATE TABLE Vet (  
  vet_ID INTEGER PRIMARY KEY,  
  vet_name VARCHAR(255),  
  clinic VARCHAR(255)  
);
```

- o **name** in **Supplies** table becomes **supply_name**

Before:

```
CREATE TABLE Supplies (  
  name VARCHAR(50),  
  branch_city VARCHAR(20),  
  branch_province VARCHAR(20),  
  quantity INTEGER,  
  PRIMARY KEY (name, branch_city, branch_province),  
  FOREIGN KEY (branch_city, branch_province) REFERENCES Branch(city,  
  province)  
  ON DELETE CASCADE  
);
```

After:

```
CREATE TABLE Supplies (  
  supply_name VARCHAR(255),  
  branch_city VARCHAR(255),  
  branch_province VARCHAR(255),  
  quantity INTEGER,  
  PRIMARY KEY (supply_name, branch_city, branch_province),  
  FOREIGN KEY (branch_city, branch_province) REFERENCES Branch(city,  
  province)
```

```
ON DELETE CASCADE  
);
```

- o **name** in **EmployeeManages** table becomes **employee_name**
Before:

```
CREATE TABLE EmployeeManages (  
  employee_ID INTEGER,  
  branch_city VARCHAR(20) NOT NULL,  
  branch_province VARCHAR(20) NOT NULL,  
  name VARCHAR(20),  
  position VARCHAR(20),  
  started_date DATE,  
  PRIMARY KEY (employee_ID),  
  FOREIGN KEY (position) REFERENCES Position(position),  
  FOREIGN KEY (branch_city, branch_province) REFERENCES Branch (city,  
  province)  
  ON DELETE NO ACTION ON UPDATE CASCADE  
);
```

After:

```
CREATE TABLE EmployeeManages (  
  employee_ID INTEGER,  
  branch_city VARCHAR(255) NOT NULL,  
  branch_province VARCHAR(255) NOT NULL,  
  employee_name VARCHAR(255),  
  position VARCHAR(255),  
  started_date DATE,  
  PRIMARY KEY (employee_ID),  
  FOREIGN KEY (position) REFERENCES Position (position),  
  FOREIGN KEY (branch_city, branch_province) REFERENCES Branch (city,  
  province)  
  ON DELETE CASCADE  
);
```

- o **size** in **FloorNumber** table becomes **cage_size**
Before:

```
CREATE TABLE FloorNumber (  
  floor_number INTEGER,  
  size VARCHAR(20),
```

```
PRIMARY KEY (floor_number)
);
```

After:

```
CREATE TABLE FloorNumber (
  floor_number INTEGER,
  cage_size VARCHAR(255),
  PRIMARY KEY (floor_number)
);
```

- o **name**, **age** and **address** in **Adopter** table become **adopter_name**, **adopter_age** and **adopter_address**

Before:

```
CREATE TABLE Adopter (
  adopter_ID INTEGER,
  name VARCHAR(20),
  age INTEGER,
  address VARCHAR(50),
  criminal_record VARCHAR(20),
  PRIMARY KEY (adopter_ID)
);
```

After:

```
CREATE TABLE Adopter (
  adopter_ID INTEGER PRIMARY KEY,
  adopter_name VARCHAR(255),
  adopter_age INTEGER,
  adopter_address VARCHAR(255),
  criminal_record VARCHAR(255)
);
```

- o **name** in **Animal** table becomes **animal_name**

Before:

```
CREATE TABLE Animal (
  animal_ID INTEGER,
  name VARCHAR(20),
```

```
breed VARCHAR(20),
age INTEGER,
admission_date DATE,
adoption_date DATE,
adopter_ID INTEGER,
cage_number INTEGER NOT NULL,
branch_city VARCHAR(20) NOT NULL,
branch_province VARCHAR(20) NOT NULL,
UNIQUE (cage_number, branch_city, branch_province),
PRIMARY KEY (animal_ID),
FOREIGN KEY (breed, age) REFERENCES AdoptionFee(breed, age),
FOREIGN KEY(adopter_ID) REFERENCES Adopter(adopter_ID),
FOREIGN KEY (cage_number) REFERENCES Cage(cage_number),
FOREIGN KEY (branch_city, branch_province) REFERENCES Branch(city,
province)
);
```

After:

```
CREATE TABLE Animal (
animal_ID INTEGER,
animal_name VARCHAR(255),
breed VARCHAR(255),
age INTEGER,
admission_date DATE,
adoption_date DATE,
adopter_ID INTEGER,
cage_number INTEGER NOT NULL,
branch_city VARCHAR(255) NOT NULL,
branch_province VARCHAR(255) NOT NULL,
UNIQUE (cage_number, branch_city, branch_province),
PRIMARY KEY (animal_ID),
FOREIGN KEY (breed, age) REFERENCES AdoptionFee(breed, age),
FOREIGN KEY(adopter_ID) REFERENCES Adopter(adopter_ID),
FOREIGN KEY (cage_number, branch_city, branch_province) REFERENCES
Cage(cage_number, branch_city, branch_province)
);
```

- **Foreign Key Modifications:** The **ON DELETE NO ACTION ON UPDATE CASCADE** clause was used for foreign key constraints in tables like **Volunteer** and

EmployeeManages. However, this clause was changed because Oracle does not support the **ON DELETE NO ACTION ON UPDATE CASCADE** option directly. Instead, these were replaced with **ON DELETE CASCADE**.

Before:

```
CREATE TABLE Volunteer (  
  volunteer_ID INTEGER PRIMARY KEY,  
  volunteer_name VARCHAR(255),  
  volunteer_role VARCHAR(255),  
  started_date DATE,  
  branch_city VARCHAR(255) NOT NULL,  
  branch_province VARCHAR(255) NOT NULL,  
  FOREIGN KEY (volunteer_role) REFERENCES Role (volunteer_role),  
  FOREIGN KEY (branch_city, branch_province)  
  REFERENCES Branch (city, province)  
  ON DELETE CASCADE  
);
```

```
CREATE TABLE EmployeeManages (  
  employee_ID INTEGER,  
  branch_city VARCHAR(20) NOT NULL,  
  branch_province VARCHAR(20) NOT NULL,  
  name VARCHAR(20),  
  position VARCHAR(20),  
  started_date DATE,  
  PRIMARY KEY (employee_ID),  
  FOREIGN KEY (position) REFERENCES Position(position),  
  FOREIGN KEY (branch_city, branch_province) REFERENCES Branch (city,  
  province)  
  ON DELETE NO ACTION ON UPDATE CASCADE  
);
```

After:

```
CREATE TABLE Volunteer (  
  volunteer_ID INTEGER PRIMARY KEY,  
  volunteer_name VARCHAR(255),  
  volunteer_role VARCHAR(255),  
  started_date DATE,  
  branch_city VARCHAR(255) NOT NULL,  
  branch_province VARCHAR(255) NOT NULL,  
  FOREIGN KEY (volunteer_role) REFERENCES Role (volunteer_role),
```



```
FOREIGN KEY (branch_city, branch_province) REFERENCES Branch (city,
province)
ON DELETE CASCADE
);
```

```
CREATE TABLE EmployeeManages (
employee_ID INTEGER,
branch_city VARCHAR(255) NOT NULL,
branch_province VARCHAR(255) NOT NULL,
employee_name VARCHAR(255),
position VARCHAR(255),
started_date DATE,
PRIMARY KEY (employee_ID),
FOREIGN KEY (position) REFERENCES Position (position),
FOREIGN KEY (branch_city, branch_province) REFERENCES Branch (city,
province)
ON DELETE CASCADE
);
```

- **Data Type Adjustment:** The **Treat** table's vaccination attribute changed from **BOOLEAN** to **INTEGER**, since Oracle does not support **BOOLEAN** data type directly, so **INTEGER** is used instead, with 0 representing NO and 1 representing YES.

Before:

```
CREATE TABLE Treat (
vet_ID INTEGER,
animal_ID INTEGER,
number_of_surgeries INTEGER,
vaccination BOOLEAN,
PRIMARY KEY (vet_ID, animal_ID),
FOREIGN KEY (vet_ID) REFERENCES Vet(vet_ID),
FOREIGN KEY (animal_ID) REFERENCES Animal(animal_ID)
);
```

After:

```
CREATE TABLE Treat (
vet_ID INTEGER,
animal_ID INTEGER,
number_of_surgeries INTEGER,
vaccination INTEGER,
PRIMARY KEY (vet_ID, animal_ID),
```

```
FOREIGN KEY (vet_ID) REFERENCES Vet(vet_ID),  
FOREIGN KEY (animal_ID) REFERENCES Animal(animal_ID)  
);
```

- **Primary Key Modification:** The **Animal** table's foreign key to **Cage** was expanded to include **branch_city** and **branch_province** along with **cage_number**. The separate foreign key referencing **Branch** (**branch_city**, **branch_province**) was removed, as these columns are already part of the **Cage** foreign key.

Before:

```
CREATE TABLE Animal (  
  animal_ID INTEGER,  
  name VARCHAR(20),  
  breed VARCHAR(20),  
  age INTEGER,  
  admission_date DATE,  
  adoption_date DATE,  
  adoper_ID INTEGER,  
  cage_number INTEGER NOT NULL,  
  branch_city VARCHAR(20) NOT NULL,  
  branch_province VARCHAR(20) NOT NULL,  
  UNIQUE (cage_number, branch_city, branch_province),  
  PRIMARY KEY (animal_ID),  
  FOREIGN KEY (breed, age) REFERENCES AdoptionFee(breed, age),  
  FOREIGN KEY(adopter_ID) REFERENCES Adopter(adopter_ID),  
  FOREIGN KEY (cage_number) REFERENCES Cage(cage_number),  
  FOREIGN KEY (branch_city, branch_province) REFERENCES Branch(city,  
  province)  
);
```

After:

```
CREATE TABLE Animal (  
  animal_ID INTEGER,  
  animal_name VARCHAR(255),  
  breed VARCHAR(255),  
  age INTEGER,  
  admission_date DATE,  
  adoption_date DATE,  
  adoper_ID INTEGER,
```

```
cage_number INTEGER NOT NULL,  
branch_city VARCHAR(255) NOT NULL,  
branch_province VARCHAR(255) NOT NULL,  
UNIQUE (cage_number, branch_city, branch_province),  
PRIMARY KEY (animal_ID),  
FOREIGN KEY (breed, age) REFERENCES AdoptionFee(breed, age),  
FOREIGN KEY(adopter_ID) REFERENCES Adopter(adopter_ID),  
FOREIGN KEY (cage_number, branch_city, branch_province) REFERENCES  
Cage(cage_number, branch_city, branch_province)  
);
```

4. SQL Queries:

- **INSERT:**

Location:

- appService.js: [Line 191](#)
- appController.js: [Line 32](#)
- scripts.js: [Line 244](#)
- index.html: [Line 27](#)

SQL Queries:

```
INSERT INTO Donate (donor_ID, branch_city, branch_province, amount)  
VALUES (:donorID, :branchCity, :branchProvince, :amount)
```

Description:

This query inserts a new row into the Donate table using user-provided values for donorID, branchCity, branchProvince, and amount. If the provided donorID does not exist in the Donors table or the specified branchCity and branchProvince do not exist in the Branch table, the query will throw an error during execution. A donor cannot donate to the same branch city and branch province twice, if we try to do the same, it results in error.

- **UPDATE:**

Location:

- appService.js: [Line 235](#), [Line 46](#), [Line 63](#), [Line 75](#)
- appController.js: [Line 71](#)
- scripts.js: [Line 296](#), [Line 134](#), [Line 15](#)
- index.html: [Line 39](#)

SQL Queries:

```
UPDATE Volunteer  
SET ${updateFields.join(', ')}  
WHERE volunteer_ID = :volunteer_id
```

Description:

This query updates specific fields for a user-chosen volunteer in the Volunteer table based on user input. The WHERE clause identifies the user-chosen volunteer by volunteer_ID, and the SET clause dynamically includes the fields the user wants to update, separated by commas. Only the specified fields are modified, while all other data remains unchanged. The changes are then immediately reflected on the table in front end.

- **DELETE:**

Location:

- appService.js: [Line 159](#), [Line 149](#)
- appController.js: [Line 44](#)
- scripts.js: [Line 38](#)
- index.html: [Line 97](#)

SQL Queries:

```
DELETE FROM Supplies
WHERE supply_name = :supplyName
AND branch_city = :branchCity
AND branch_province = :branchProvince
```

Description:

This query deletes entries from the Supplies table based on user-provided inputs for supply_name, branch_city, and branch_province. The rows matching these criteria are removed from the table. Additionally, if a branch city and branch province is deleted from the Branch table, the ON DELETE CASCADE takes care such that all entries with the corresponding branch city and branch province values in the Supplies table is also deleted.

- **SELECTION:**

Location:

- appService.js: [Line 507](#)
- appController.js: [Line 278](#)
- scripts.js: [Line 628](#)
- animalSearch.js: [Line 66](#)
- index.html: [Line 219](#)
- animalSearch.html: [Line 29](#)

SQL Queries:

```
SELECT *
FROM Animal
WHERE ${whereClause}
```

Description:

This query retrieves rows from the Animal table that match the user-defined conditions. The conditions can involve comparisons such as equality, inequality, greater than, or less than. For example, if the user specifies animal_ID = 1001 OR age = 2, the query returns rows where animal_ID is 1001 or age is 2.

- **PROJECTION:**

Location:

- appService.js: [Line 434](#), [Line 28](#)
- appController.js: [Line 266](#), [Line 198](#)
- scripts.js: [Line 541](#), [Line 631](#), [Line 501](#)
- index.html: [Line 224](#)

SQL Queries:

```
CREATE VIEW selectedColumns AS
SELECT ${columns.join(', ')}
FROM Adopter
```

Description:

This query allows the user to select specific attributes from the Adopter table. By specifying the desired columns, the result displays only the chosen attributes. For instance, if the user selects name and age, the output will include only those columns.

- **JOIN:**

Location:

- appService.js: [Line 468](#)
- appController.js: [Line 229](#)
- scripts.js: [Line 571](#)
- index.html: [Line 193](#)

SQL Queries:

```
SELECT donor_name, donated_item
FROM Donor, Donate
WHERE Donor.donor_ID = Donate.donor_ID AND branch_city = :branch_city
AND branch_province = :branch_province
```

Description:

This query retrieves data from the Donor and Donate tables by combining them based on matching donor_ID values. Additionally, it filters results by branch_city and branch_province as specified by the user. For example, it displays donor_name and donated_item for donors in a specific city and province.

- **Aggregation with GROUP BY:**

Location:

- appService.js: [Line 297](#)

- appController.js: [Line 100](#)
- scripts.js: [Line 375](#)
- index.html: [Line 156](#)

SQL Queries:

```
SELECT COUNT(*) AS donation_count
FROM Donate
WHERE branch_city = :branch_city
AND branch_province = :branch_province
GROUP BY branch_city, branch_province
```

Description:

This query counts the total number of donations for a specific branch, identified by its city and province. It demonstrates aggregation with GROUP BY counting number of donations for a specific branch. It uses COUNT(*) to aggregate the number of donations, grouping results by branch_city and branch_province. The WHERE clause filters for a particular branch, allowing for dynamic input through parameters.

- **Aggregation with HAVING:**

Location:

- appService.js: [Line 330](#)
- appController.js: [Line 125](#)
- scripts.js: [Line 405](#)
- index.html: [Line 168](#)

SQL Queries:

```
SELECT branch_city, branch_province, SUM(amount) AS total_donations
FROM Donate
GROUP BY branch_city, branch_province
HAVING SUM(amount) > 50
```

Description:

This query identifies branches where the total donation amount exceeds \$50. It groups donations by branch location and uses the HAVING clause to filter results based on the aggregated sum, showcasing branches with significant donation totals.

- **Nested aggregation with GROUP BY:**

Location:

- appService.js: [Line 357](#)
- appController.js: [Line 148](#)
- scripts.js: [Line 436](#)
- index.html: [Line 177](#)

SQL Queries:

```
SELECT branch_city, branch_province, AVG(amount) AS avg_donation
FROM Donate
```

```
GROUP BY branch_city, branch_province  
HAVING AVG(amount) > (SELECT AVG(amount) FROM Donate)
```

Description:

This query compares each branch's average donation amount to the overall average across all branches. It returns only those branches whose average donation exceeds the overall average, highlighting locations with above-average donor generosity.

- **DIVISION:**

Location:

- appService.js: [Line 384](#)
- appController.js: [Line 172](#)
- scripts.js: [Line 467](#)
- index.html: [Line 185](#)

SQL Queries:

```
SELECT DISTINCT D1.branch_city, D1.branch_province  
FROM Donate D1  
WHERE NOT EXISTS (SELECT donor_ID  
                  FROM Donate D2  
                  WHERE NOT EXISTS(SELECT 1  
                                   FROM Donate D3  
                                   WHERE D3.donor_ID = D2.donor_ID  
                                   AND D3.branch_city = D1.branch_city  
                                   AND D3.branch_province =  
D1.branch_province))
```

Description:

This query will return the branches which received donations from all donors in the record. This query showcases the division operation in relational algebra. It identifies branches that have received donations from all donors in the database. The division is implemented using nested NOT EXISTS subqueries, effectively finding the set of branches (divisor) that have received donations from every donor (dividend) in the system.